available that act as GUIs for *iptables*, like XFWall. You can use these tools to control your network traffic. XFWall is a very good piece of software, but not entirely documented. Another such software is Firewall Builder, which is also very good (some say the best). However, let's not forget that firewall configuration is unique for every network.

There is another simple, but expensive solution. It constitutes buying an ADSL router (modem), which has in-built firewalling capabilities. My aim was to make the server simple to maintain, and that involves not writing long scripts. If you have a Type-1 ADSL modem (1 port each for power, a phone-line and the network, respectively), then a separate router would be required anyway, because you require port forwarding, which is not present in Type 1 routers. If you go for this, I'd suggest one from Linksys (Cisco)--they're quite easily available.

Network Address Translation

NAT, or Network Address Translation, is a protocol used to forward IP packets from one interface to another. This is a bit different from bridging. Anyway, NAT is essential if we want to browse the Internet from a client. To enable it, execute the following:

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
# iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
# iptables -A FORWARD -i eth0 -o eth1 -m state \
--state RELATED,ESTABLISHED -j ACCEPT
#: iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
```

In fact, it's better you add these lines to the end of your */etc/rc.local* file to set up NAT every time your server boots.

Note: If the system cannot find the *iptables* command, add an */sbin/* before it. If that too doesn't work, you need to go for an *apt-get install*.

Port forwarding

This is another feature about which I cannot help you much. Port forwarding configuration is different for every modem. Anyway, the instructions here are a bit more general.

Only the ADSL modem can be seen by people on the Internet, not your server. Because of this, all traffic stops at your modem. Port forwarding is enabled to forward all data packets from the modem to the respective port on your computer. If an HTTP request is sent to port 80 of the modem, the request is forwarded to port 80 on your server, on which (hopefully) your HTTP server is running.

You can avoid port forwarding altogether if you do not have an always-on connection. I recommend you go for it, but you don't have to. Make a call to your Internet service provider requesting your modem to be switched back to bridged mode. Use the static IP address you obtained from your ISP here. Now execute *pppoeconf* from your server. Configure the connection. When you are done, run *pon dsl-provider* to bring up the Internet connection. That's it!

DHCP, PXE, DNS and TFTP

Now we begin with the juicy part: setting up the server. Before you start, make sure a client is connected to your server. For simplicity, go for a Linux client.

Issue this command: *sudo apt-get install dnsmasq*. This will install an all-in-one DNS, DHCP and TFTP server on your computer.

Before we start with DHCP, we need to configure the DNS forwarder. The configuration for DNSMASQ is stored in the */etc/dnsmasq.conf* file. We need to edit it. But first, execute these commands:

```
#: mkdir /etc/dnsmasq
#: mv /etc/resolv.conf /etc/dnsmasq/resolv.upstream
#: echo "nameserver 127.0.0.1" > /etc/resolv.conf
```

This moves the previous */etc/resolv.conf* file to */etc/resolv.upstream* and creates a new *resolv.conf* that references the locally running DNSMASQ. Right now, Internet browsing should not be working.

Open the */etc/dnsmasq.conf* file for editing in a text editor and set the following parameters (uncomment them if necessary):

1. First of all, set it to listen on 'loopback' and 'eth1'. Do this by setting two interface lines: